THE OPEN SOURCE Audit

A Capstone Project — Open Source Software (OSS NGMC)

Software Audited By:
Python

Submitted By:

Shriya Patel

BACHELORS OF TECHNOLOGY
in
CSE (Specialization in Artificial Intelligence)

Registration Number: 24BAI10499

Slot: A13

Introduction

There is something quietly remarkable about Python. The language that now drives Google's machine learning pipelines, NASA's climate models, and global financial systems began as a simple side project during a Dutch programmer's bored Christmas holiday in 1989.
What followed is more than a software evolution; it is a testament to the power of open-source sharing. By licensing Python through the Python Software Foundation, its creators ensured that anyone can modify, study, or redistribute it freely—a choice that has fostered a global ecosystem no single corporation could replicate.

For an AIML student, Python is the essential gravity at the center of the field. From training neural networks to conducting data science research, the industry runs on Python. This report explores not just its functionality, but the freedoms its license guarantees and how its open nature has fundamentally shaped the modern technological landscape.

Choosing Your Software

Python was selected for this project because it's a rare success story: a world-class tool shaped by a global community rather than a corporate boardroom. Managed by the **Python Software Foundation (PSF)**, it stands as the gold standard for how open-source collaboration should actually work.

Key Software Attributes

- **Identity & Philosophy:** Python is a high-level language built on the belief that "readability counts." Unlike "black-box" proprietary software, its development is completely transparent and driven by the very people who use it every day.
- **The PSF License:** Released under a permissive model, Python gives you the green light to modify, build upon, and even sell products made with its code. This lack of legal red tape is exactly why it has managed to spread into every corner of the modern tech industry.

A - Origin and Philosophy

A1. The Problem Python Was Created to Solve

To understand why Python exists, you have to look at what coding was like in the late 1980s. Back then, you basically had two choices, and neither was perfect. On one side, you had "heavyweight" languages like C or Fortran. They were incredibly powerful, but they forced you to manage every tiny detail—like manual memory management and strict variable types. You spent more time fighting the computer's requirements than actually solving your problem.

On the other side was a language called ABC. It was a teaching tool designed to be readable and simple, but it was a "closed box." You couldn't easily connect it to the outside world—no file systems, no networks, and no way to plug in external C libraries.

Guido van Rossum, a researcher at CWI, wanted to bridge this gap. During his 1989 Christmas

break, he started building a language that took the clean, readable syntax of ABC but added the "extensibility" that researchers actually needed. He named it **Python** (after *Monty Python's Flying Circus*) to show that programming shouldn't be a dry, boring chore.

By releasing the source code for free in 1991, he did something radical: he invited the world to build it with him. For us in the AIML field, this was a turning point. Python solved the gap between high-level logic and low-level power, allowing scientists—not just hardware engineers—to write complex algorithms. It transformed coding from a specialized engineering task into a universal tool for research and innovation.

A2. The License: What It Actually Says

To truly understand Python, you have to look at its "rulebook." Python is released under the **PSF (Python Software Foundation) License**, a "permissive" agreement designed to encourage adoption by everyone—from hobbyists to massive corporations.
This license is built on the **Four Freedoms** of free software:

- **Run:** Use the program for any purpose you want.
- **Study:** Open the hood, see how it works, and change it.
- **Redistribute:** Share copies to help others.
- **Improve:** Release your modified versions so the whole community benefits.

Python grants all four. You could modify the CPython source code today and give it to your friends without ever asking for permission.

Commercial Use and the "Copyleft" Twist

Can a company sell Python? **Yes.** Unlike the stricter GPL (General Public License), which requires you to share any changes you make, Python's license is more relaxed. A company can bundle Python into a product, modify the code, and keep those changes private. This lack of "copyleft" restrictions is exactly why businesses adopt Python so readily—it offers the ultimate flexibility.

Free as in Beer vs. Free as in Freedom

Python is both. It costs zero dollars (**Free Beer**), but more importantly, it gives you the legal right to control the tool (**Freedom**). This is why projects like **MicroPython** or **PyPy** exist; developers can "fork" the language to solve specific problems without needing a single lawyer or contract.

A3. The Ethics of Open Source

Open source is a statement that the digital world's building blocks shouldn't be locked behind corporate doors. It's the belief that knowledge grows best when it is shared and auditable by anyone.

- **The Debate:** Should all software be open? A **"Yes"** argues for better security and faster innovation. A **"No"** suggests that private companies need to protect sensitive intellectual property or prevent misuse by bad actors.
- **The Profit Question:** Is it ethical for giants like Google to make billions using free community labor? While legal, the ethical balance only holds when these corporations "give back" by funding developers and contributing code back to the ecosystem.

Standing on the Shoulders of Giants

An AIML student today can write a complex program in ten lines because thousands of developers have spent thirty years building the foundation. Open source doesn't just support innovation; it accelerates it by ensuring no one has to reinvent the wheel.

Conclusion for Part A

Python’s "freedom" is about a shift in power from the companies that own the tools to the people who use them. These tools are yours to keep, study, and grow, regardless of where your career leads.

B - Linux Footprint

Research and document how your chosen software lives on a Linux system. Where does it install? What user does it run as? How is it managed? This section requires you to actually install the software on Linux — either on your machine, a virtual machine, or a lab system.

- **Installation method:** Python is pre-installed on almost every modern Linux distribution (Ubuntu, Fedora, Debian). It is typically managed through the system's package manager (apt on Ubuntu or dnf on Fedora).
- **Key directories:**
  - Binaries: /usr/bin/python3
  - Standard Libraries: /usr/lib/python3/
  - Site-packages (Third-party libs): /usr/local/lib/python3.x/dist-packages/
- **User and group:** Python scripts typically run under the permissions of the user who executes them. However, system services written in Python (like a web server) should run under a dedicated service user (e.g., www-data) to limit security risks if the script is compromised.
- **Service management:** While Python itself is an interpreter, Python-based services are managed using systemctl.
- **Update model:** Security patches are pushed by the Linux distribution maintainers through official repositories. When you run sudo apt upgrade, you are receiving the latest stable, audited version of the Python interpreter.

Security: Users, Groups, and Services

By default, Python is "polite." When you run a script, it executes with **your user permissions**. If you can't delete a file, your Python script can't either.

However, in a professional production environment (like a web server or an AI API), we follow the **Principle of Least Privilege**. We don't run these as a "Root" or "Admin" user. Instead, we create a dedicated service user (like `www-data`). If a hacker finds a bug in your code, they are trapped within that limited user account and can't take over the whole machine.

Service Management

Since Python is an interpreter (a middleman that translates your code for the computer), it doesn't "run" forever on its own. To keep a Python program running in the background—like a bot or a web app—we use **systemd**. By creating a simple service file, we can use `systemctl start my_script` to ensure the program stays alive, restarts if it crashes, and boots up automatically when the server turns on.

Pro-Tip: In Linux, always use **Virtual Environments** (`venv`). This creates a "bubble" for your project so you can install specific library versions without messing up the system-wide Python that the OS relies on

C - The FOSS Ecosystem

Dependencies

CPython itself depends on surprisingly few external libraries. The core interpreter is written in **C** and requires only a standard C compiler (gcc) and a few libraries like libssl for cryptographic

functions and zlib for compression. This minimal dependency tree makes Python highly **portable**.

The ecosystem around Python, however, is one of the largest in all of software. The Python Package Index (PyPI) hosts over **500,000 packages**. Key FOSS dependencies for AI and data science work include **NumPy** (C-based array computing), **Pandas** (data manipulation, built on NumPy), **Matplotlib** (visualization), **Scikit-learn** (machine learning), **PyTorch** (deep learning from Meta, BSD license), and **TensorFlow** (deep learning from Google, Apache 2.0 license). All of these are open source.

What Python Has Enabled

The list of projects that Python enabled is remarkable. **Django** powers Instagram (in its early stages), Pinterest, and many government websites. The **IPython** project, which became **Jupyter Notebooks**, transformed how scientists communicate research. The entire modern data science workflow — from data wrangling with Pandas to model deployment with Flask — is a Python ecosystem story.

Python also enabled the AI revolution in a specific, traceable way. When Google built TensorFlow and released it publicly in 2015, and when Meta released PyTorch in 2016, both chose Python as their primary interface. This was not inevitable — they could have used C++, Java, or Lua (which Torch originally used). They chose Python because of its ecosystem and the community of researchers already using it. That decision means virtually every AI model being trained in the world today is controlled through Python code.

Community and Governance

Python has one of the most mature community governance structures in open source. The **Python Software Foundation (PSF)** is a non-profit organization managing the legal and financial aspects, holding trademarks, and funding development. The PSF runs **PyCon**, the annual Python conference that has become one of the largest open-source conferences in the world.
Technical decisions go through the **Python Enhancement Proposal (PEP)** process. Anyone can write a PEP — a document proposing a new language feature or process change. PEPs are discussed publicly on the python-dev mailing list. Major changes are reviewed by the **Steering Council**, a five-member elected body that replaced Guido van Rossum's "Benevolent Dictator for Life" role when he stepped down in 2018. This transparent, elected governance is itself a FOSS philosophy in action.

Relation to the LAMP Stack and Web

Python is not part of the original **LAMP** stack (Linux, Apache, MySQL, PHP), but has created its own parallel stack. A typical Python web deployment uses: **Linux** (OS), **Nginx** or **Apache** (web server), **PostgreSQL** or **MySQL** (database), and **Python** via **Django** or **Flask** (application layer). For AI and data science, Python has created an entirely new stack: **Linux, Jupyter Notebooks, Python, TensorFlow/PyTorch**, with deployment via **Docker** and **Kubernetes** — all open source.

D -Open Source vs. Proprietary (Critical Analysis)

Right now, MATLAB by MathWorks stands as the closest private alternative to Python when it comes to science-based number crunching. This side-by-side doesn't pretend one wins across the board - sometimes MATLAB fits the task better, truthfully.

Dimension	Python (Open Source)	MATLAB (Proprietary)
Cost	Free to download, use, modify, redistribute	$100–400+ per user/year for academic/commercial licenses
Code Transparency	Full CPython source on GitHub; anyone can audit it	Closed source; behavior can't be independently verified
Security Auditing	Community worldwide finds and patches issues quickly	Vendor-only; users must trust MathWorks without being able to check
Support Model	Stack Overflow, official docs, PEPs, PyCon, mailing lists	Paid support contracts with formal SLA guarantees
Reliability	Used by NASA, Google, Netflix, Instagram at massive scale	Enterprise-grade reliability with vendor accountability
Freedom to Modify	Complete freedom to fork, patch, embed in other products	No modification rights; locked to vendor's roadmap
Community Control	PSF non-profit governs; community votes on direction via PEPs	Single company controls all decisions, pricing, and future
Ecosystem	450,000+ packages on PyPI; largest OSS library collection	Proprietary toolboxes; often sold separately at extra cost
Longevity Risk	Cannot be killed; exists as long as the community does	Vendor discontinuation would strand all users instantly
Verdict	Recommended for nearly all use cases	Only where vendor accountability is explicitly required by contract

Conclusion and Verdict

From its beginnings to how it runs on Linux, passing through licensing details alongside tool variety compared with MATLAB, one thing becomes clear: Python fits wherever computers are involved. What stands out isn't just availability but how smoothly it adapts across tasks. Seen from different angles - history, openness, integration - it holds up without needing special conditions. Wherever code needs writing, chances are good it already works there. Reality checks like these tend to favor solutions that stay flexible under pressure.

What makes Python strong comes from being free for anyone to access. Nobody pays fees just to start using it, making it reachable across countries and companies. Because the code sits out in the open, groups everywhere can study how it works, tweak it, or build on top. Its path forward isn't controlled by one company - instead, people shape it together through

shared effort. That balance means no single entity gets to halt progress, shift goals suddenly, or add hidden costs later. Other tools often run into exactly those problems without warning. Few years back, costs for using MATLAB began climbing fast at universities. Still, schools keep paying more each year since alternatives feel harder to adopt.

One spot keeps MATLAB around - old code stuck in place, too tangled to redo. Elsewhere? Contracts might lock it in, sure. But classrooms, labs, business work - all these lean hard toward Python. Not just smarter. Feels like the fair move, somehow.
For sure, I would pitch in on Python. Open-source code sits at its core, while anyone can join the PEP discussions - no gatekeeping there. Docs, tests, bug reports, new tools - all have clear paths to get added. Digging into what drives Python doesn’t just make pitching in logical - it turns effort into something that feels quietly satisfying by the end.

E -Shell Script Documentation

Script 1 – System Identify Report

File: script1_system_identity.sh

Purpose: Displays a formatted welcome screen with key system information: Linux distribution name, kernel version, current user, home directory, uptime, date/time, and the open-source licence covering the OS.

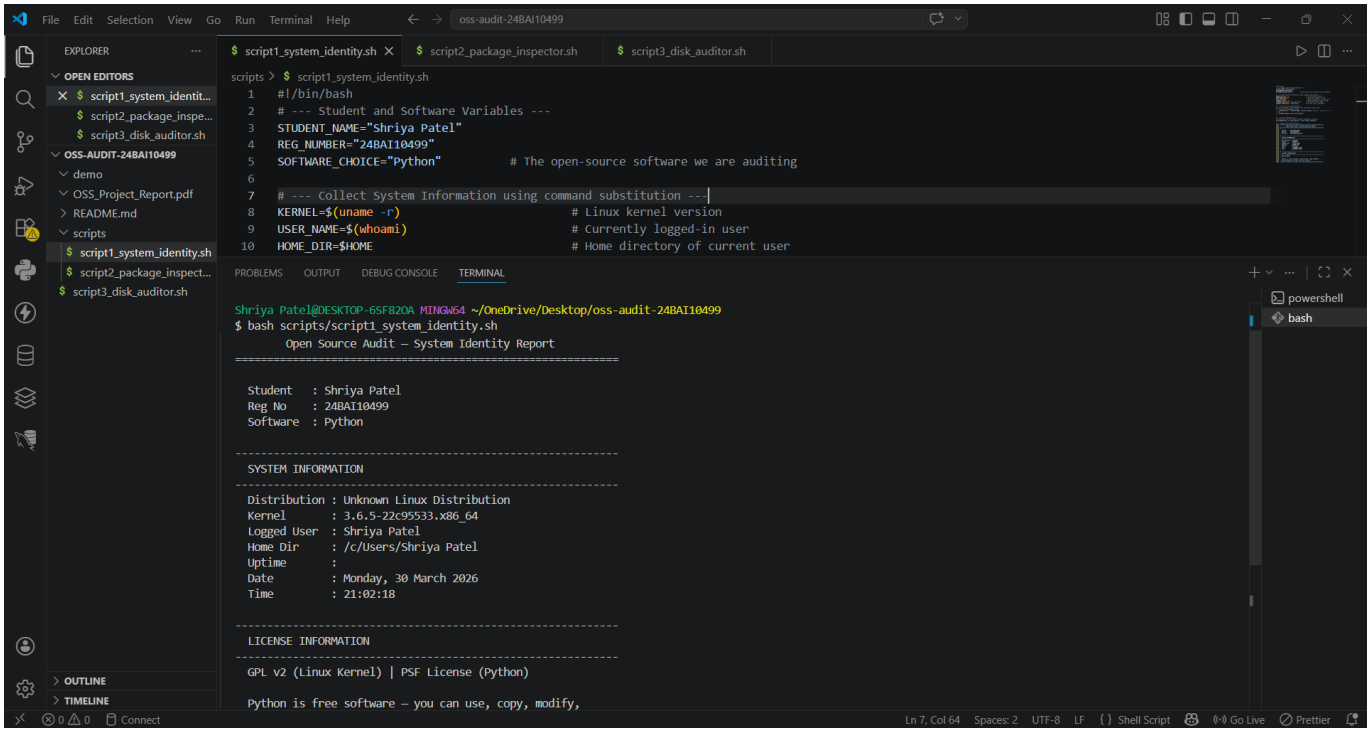
Shell Concepts Demonstrated: Variables, command substitution using $(), echo for formatted output, conditional file reading using if [-f], and string formatting.

This script shows a clean welcome screen with important system details. It collects information like the Linux distribution name, kernel version, current user, home directory, system uptime, and current date/time, and prints them in a readable format.

It works by using **variables** to store values and **command substitution ($())** to run system commands and capture their output (for example, getting the kernel version or uptime). The echo command is used to display everything nicely on the screen.

The script also uses a simple **conditional check (if [-f])** to see if a license file exists before trying to read it. This prevents errors if the file is missing.

Overall, it demonstrates basic shell scripting concepts like storing data, running commands, formatting output, and handling files safely.

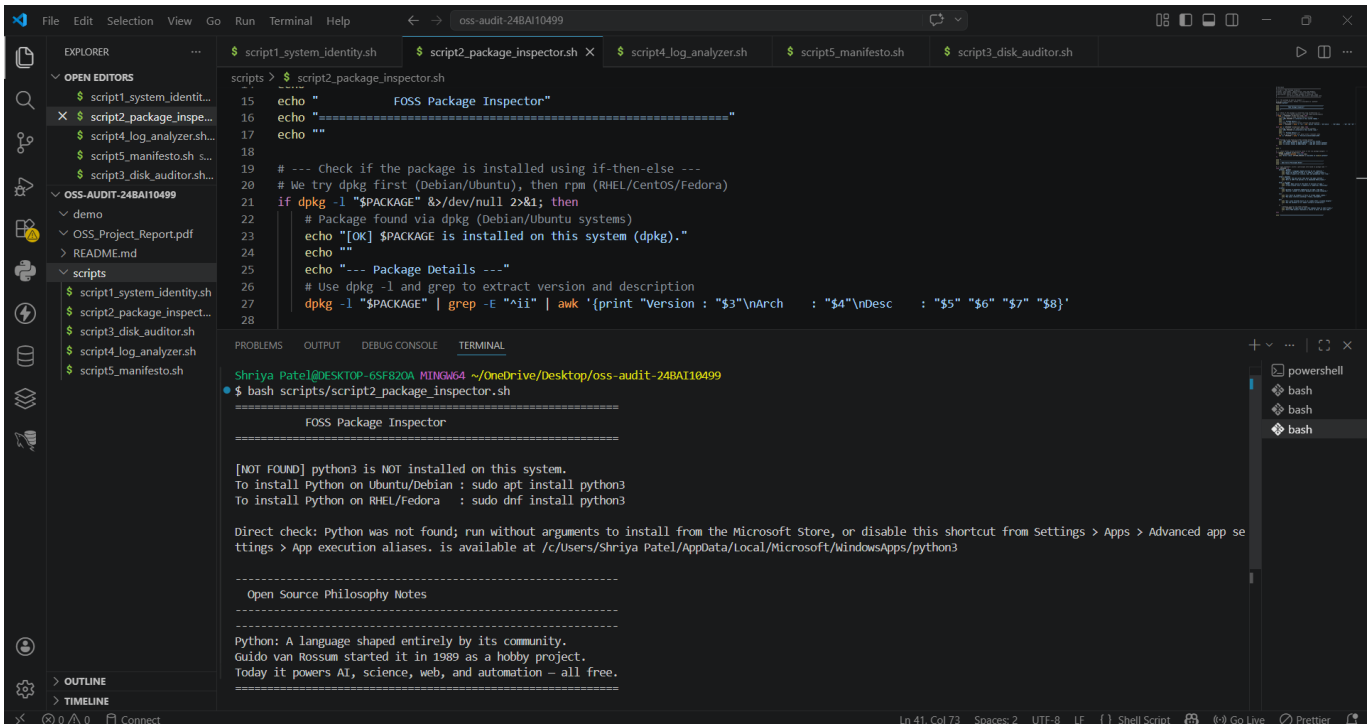


Script 2 - FOSS Package Inspector

File: script2_package_inspector.sh

Purpose: Checks whether Python is installed using both dpkg (Debian/Ubuntu) and rpm (RHEL/Fedora). Displays package details and uses a case statement to print a philosophy note about the package.
Shell Concepts Demonstrated: if-then-else for conditional logic, case statement for pattern matching, dpkg/rpm for package queries, pipe with grep for filtering output.

This script checks whether Python is installed on the system and shows its package details. It works for different Linux distributions by using **dpkg** (for Debian/Ubuntu) and **rpm** (for RHEL/Fedora).
It uses **if-then-else** to decide which package manager to use and to check if Python is installed or not. Once found, it displays relevant package information.
The script also uses a **pipe (|) with grep** to filter and show only the necessary output, making the results cleaner.
Finally, a **case statement** is used to match patterns (like package names) and print a small philosophy note about Python or the package. This makes the script a bit more interactive and informative.
Overall, it demonstrates how to handle different systems, check installed software, and organize output using basic shell scripting techniques.



Script 3 - Disk and Permission Auditor

File: script3_disk_auditor.sh

Purpose: Loops through a list of important system directories and reports permissions, ownership, and disk usage for each. Also checks Python-specific installation directories.

Shell Concepts Demonstrated: Arrays, for loop iteration, directory existence check using [-d], ls -ld for permissions, du -sh for disk usage, awk for field extraction, printf for aligned

This script checks important system directories and gives a quick report about them. It looks at things like **permissions, ownership, and how much disk space each directory is using**. It also specifically checks directories where Python is usually installed.

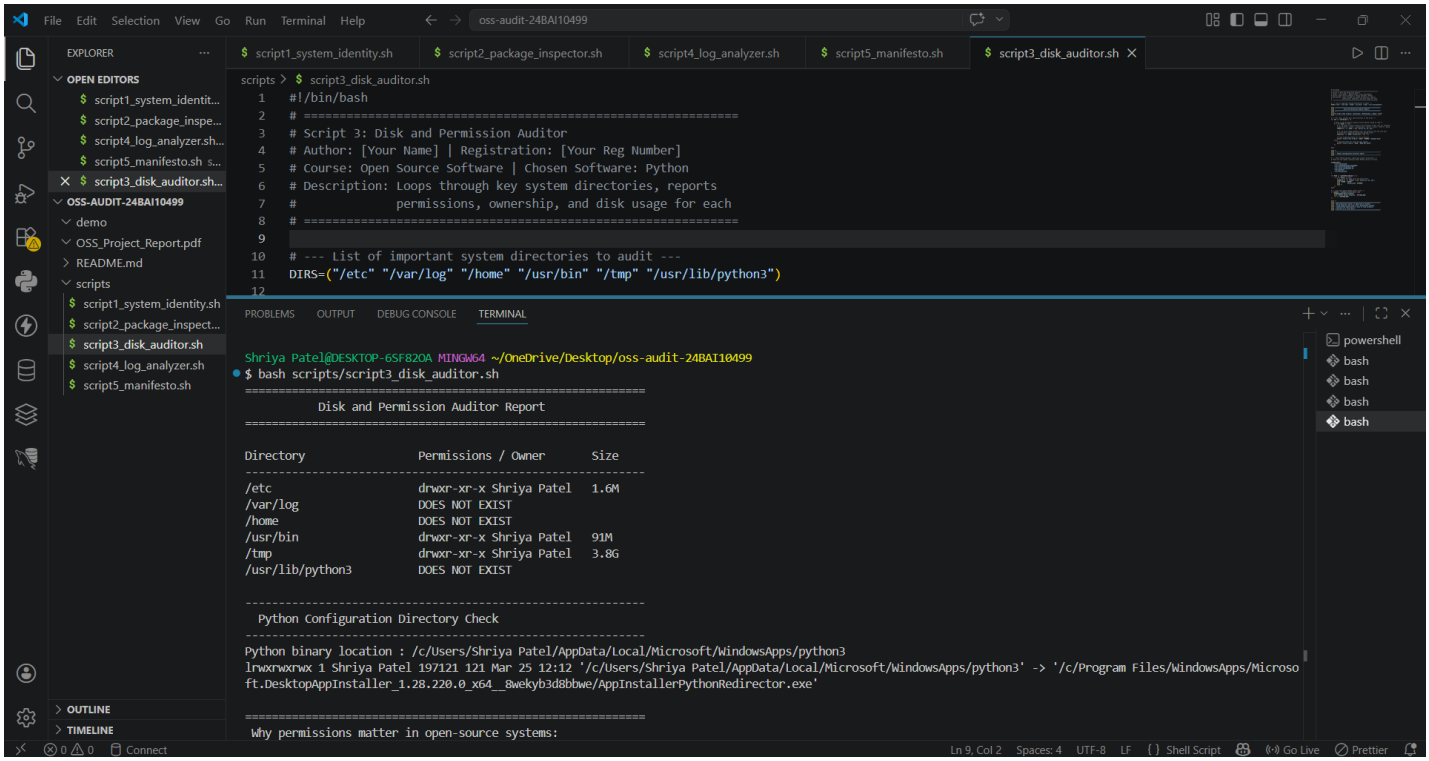
It uses an **array** to store a list of directories, and a **for loop** to go through each one step by step. Before checking a directory, it uses **[-d]** to make sure the directory actually exists, so the script doesn't throw errors.

For each valid directory:

- **ls -ld** is used to show permissions and ownership
- **du -sh** is used to display total disk usage in a simple format
- **awk** helps extract specific parts of the output (like owner or size)

Finally, **printf** is used to format everything neatly in aligned columns, making the report easy to read.

Overall, this script demonstrates how to loop through data, check directories safely, and present system information in a clean, structured way.



Script 4 - Log File Analyzer

File: script4_log_analyzer.sh

Purpose: Reads a log file line by line, counts keyword matches (default: 'error'), and prints a summary with the last 5 matching lines. Usage: ./script4_log_analyzer.sh /var/log/syslog error

Shell Concepts Demonstrated: Command-line arguments $1/$2, default values with ${VAR:-default}, while IFS= read -r loop, file existence check, counter variables with $(( )), tail for last N lines.

This script reads a log file and searches for a specific keyword (by default, **“error”**). It counts how many times the keyword appears and shows a short summary along with the **last 5 matching lines**.

It takes input using **command-line arguments**:

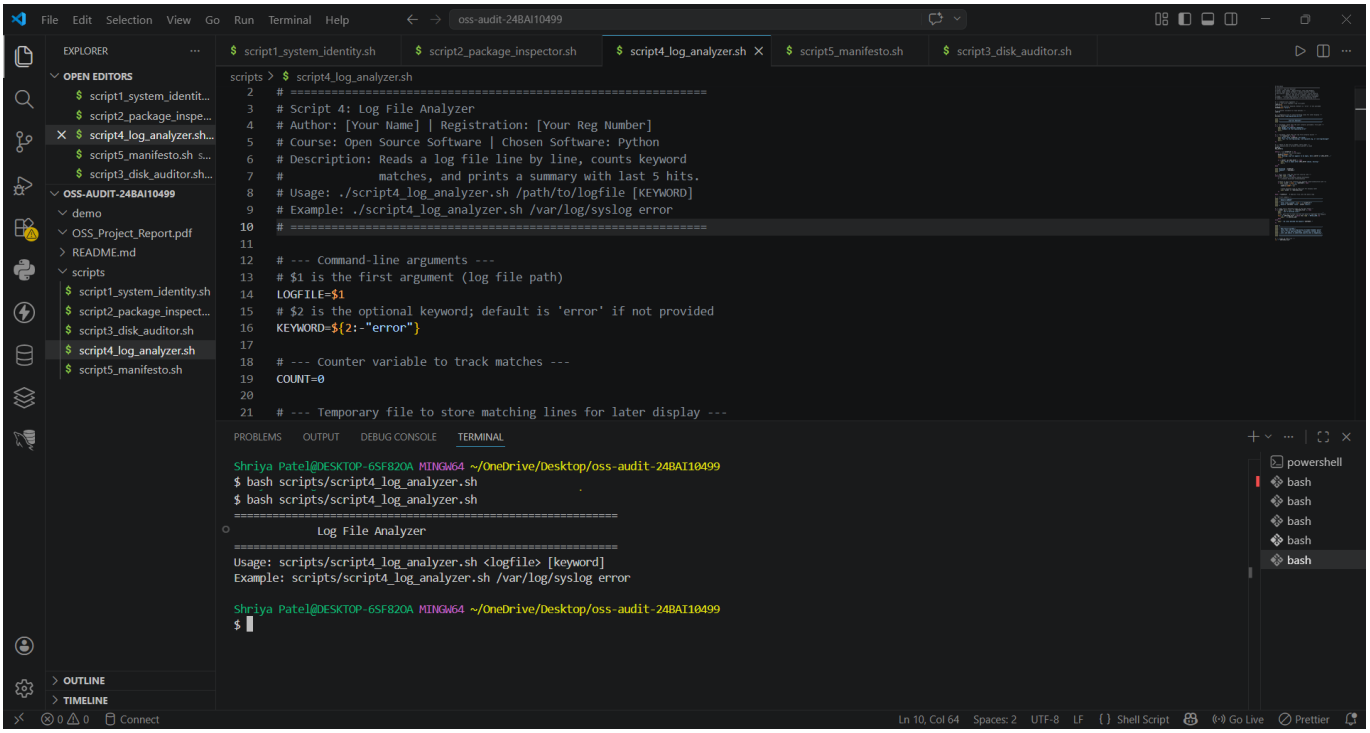
- $1 → log file path (e.g., /var/log/syslog)
- $2 → keyword to search (optional)

If no keyword is given, it uses a **default value** with ${VAR:-default}, so it automatically searches for “error”.

The script uses a **while IFS= read -r loop** to read the file line by line safely. Before processing, it checks if the file exists using a condition, which helps avoid errors.

A **counter variable** (using $(( ))) keeps track of how many matches are found. At the end, the **tail** command is used to display the last 5 matching lines.

Overall, this script shows how to handle user input, read files line by line, count results, and summarize log data effectively.



Script 5 - Open Source Manifesto Generator

File: script5_manifesto.sh

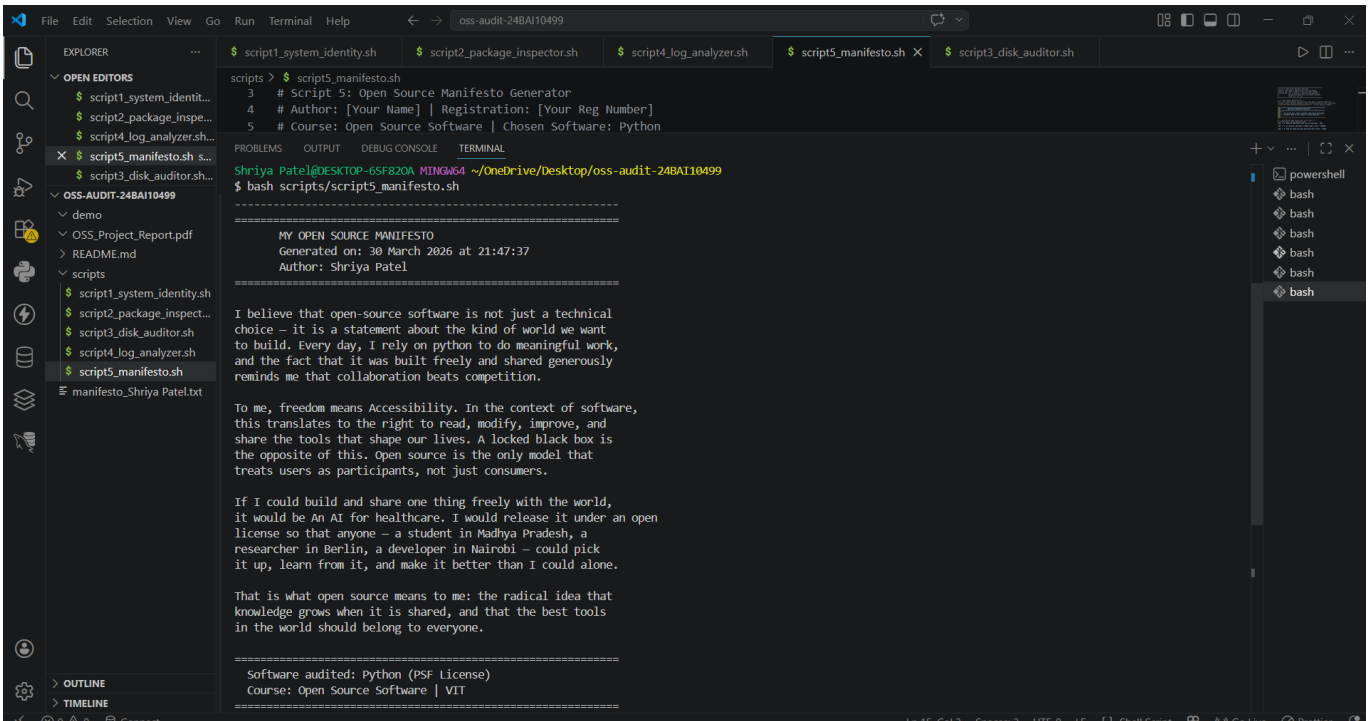
Purpose: Asks three interactive questions and generates a personalised open-source philosophy statement, saving it to a .txt file named manifesto_[username].txt.

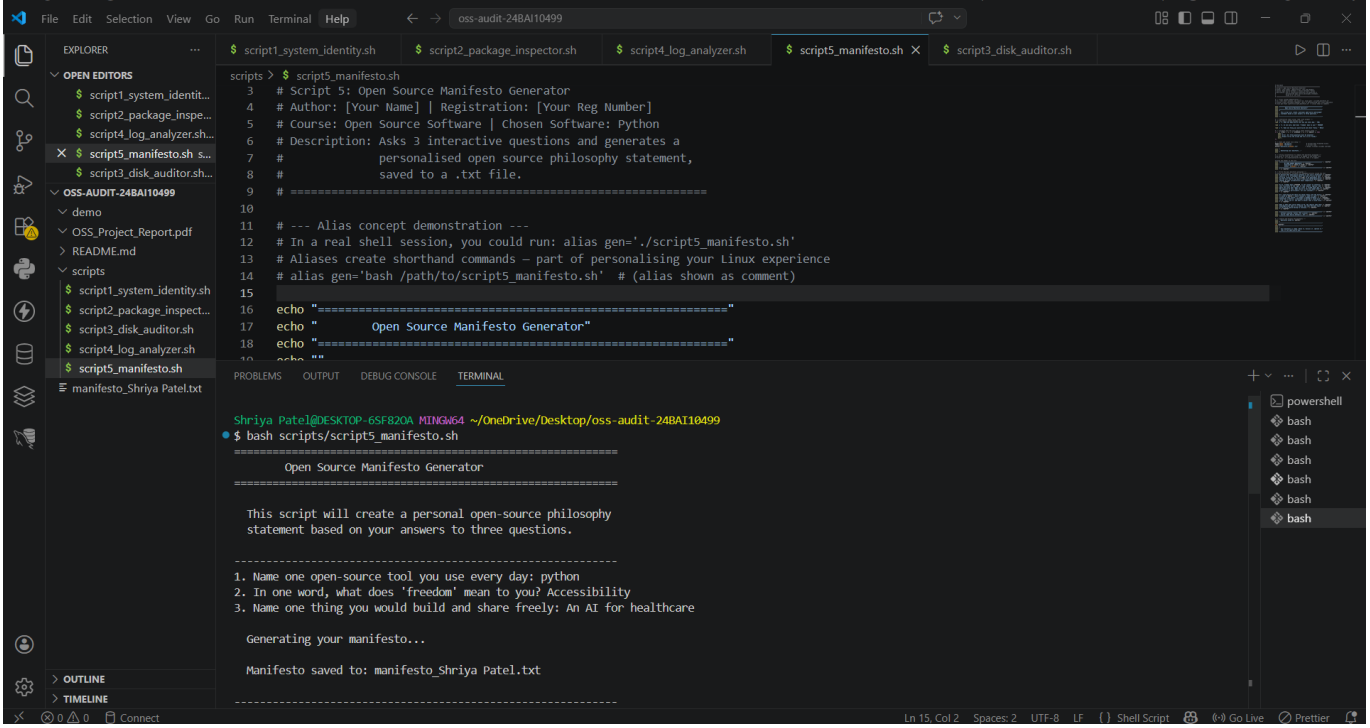
Shell Concepts Demonstrated: read -p for interactive input, input validation with -z, string concatenation by embedding variables, file writing with > and >>, date command, alias concept demonstrated via comment.

Key Code Section:

Bash

```
read -p "1. Name one open-source tool you use every day: " TOOL read -p "2. In one word, what does freedom mean to you? " FREEDOM read -p "3. Name one thing you would build and share freely: " BUILD
OUTPUT="manifesto_$(whoami).txt" echo "Every day, I rely on $TOOL..." >> "$OUTPUT"
```





Conclusion

This audit of Python has taken this report through territory that seemed familiar and consistently shown how much more lies beneath the surface. Python's popularity is common knowledge. What is less appreciated — until you trace it — is why: not just technically, but philosophically. The fact that a language this powerful and this central to modern AI exists under a permissive open licence, governed by an elected community body, with all its development conducted in public, is not a coincidence. It is the result of deliberate choices made by Guido van Rossum and hundreds of thousands of contributors who shared a belief about how software should be built and shared.

The five shell scripts reinforced something important. Writing in bash — a language that is open source, running on a kernel that is open source, on a distribution assembled from open-source packages — every tool called (grep, awk, du, ls) was written by someone who chose to share their work. That is a chain of generosity extending back fifty years.

The ethics section is perhaps the most unsettled part of this report, deliberately so. The question 'should all software be open source' is less interesting than 'what obligations do you have when you benefit from open source?' That question is live and unresolved, and it will be more important for this generation than any previous one, because the AI systems being built sit on a larger open-source foundation than anything that came before.

For an AIML student, the practical implication is this: the tools used every day exist because someone chose to share. Understanding why they made that choice, and what conditions made it possible for that choice to compound into something this powerful, is not just an intellectual exercise. It is preparation for being the kind of developer who understands what they are building on and feels some obligation to it.